

Pointer and Reference

林致宇

Outline

- Pointer
- Pointer and Array
- Dynamic Memory Allocation
- Smart Pointer
- Reference
- The applications of Pointer and Reference in Functions

Outline

- **Pointer**
- Pointer and Array
- Dynamic Memory Allocation
- Smart Pointer
- Reference
- The applications of Pointer and Reference in Functions

Introduction

- CPU & Memory
- Memory Address

Pointer Variables

```
lincyu@Chih-YudeMacBook-Pro Programs % ./a.out
0x16d34784c, 5, 8
0x16d347840, 3, 8
3, 8
0x16d347838, 0x16d347830
```

- `DataType * VariableName;`
- `&` address-of operator (取址運算子)

```
4  int main(int argc, char * argv[]) {
5
6      int i = 5;
7      double d = 3.0;
8
9      int * pi = &i;
10     double * pd = &d;
11
12     cout << pi << ", " << *pi << ", " << sizeof(pi) << '\n';
13     cout << pd << ", " << *pd << ", " << sizeof(pd) << '\n';
14     cout << d << ", " << sizeof(d) << '\n';
15
16     cout << &pi << ", " << &pd << '\n';
17 }
```

Pointer Variables

(pd) 0x16d347830

0x16d347840

(pi) 0x16d347838

0x16d34784c

(d) 0x16d347840

3.0

(i) 0x16d34784c

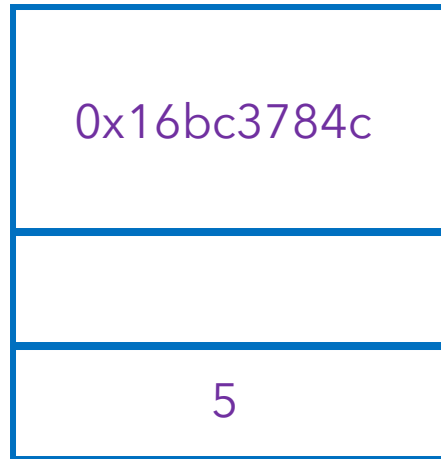
5

```
lincyu@Chih-YudeMacBook-Pro Programs % ./a.out
0x16d34784c, 5, 8
0x16d347840, 3, 8
3, 8
0x16d347838, 0x16d347830
```

```
4  int main(int argc, char * argv[]) {
5
6      int i = 5;
7      double d = 3.0;
8
9      int * pi = &i;
10     double * pd = &d;
11
12     cout << pi << ", " << *pi << ", " << sizeof(pi) << '\n';
13     cout << pd << ", " << *pd << ", " << sizeof(pd) << '\n';
14     cout << d << ", " << sizeof(d) << '\n';
15
16     cout << &pi << ", " << &pd << '\n';
17 }
```

Pointer Variables

(px) 0x16bc37840



&

Address-of operator

*

Indirect operator / Dereference operator

```
4  int main() {  
5  
6      int x = 5;  
7  
8      int * px = &x;  
9  
10     cout << px << ", " << &px << "\n";  
11     cout << *px << ", " << *(&px) << "\n";  
12 }
```

老師提問：
*(&px) 是多少？

```
lincyu@Chih-YudeMacBook-Pro Programs % ./a.out  
0x16bc3784c, 0x16bc37840  
5, 0x16bc3784c  
lincyu@Chih-YudeMacBook-Pro Programs %
```

const

- A pointer to a constant
- A constant pointer
- A constant pointer to a constant

A pointer to a constant

```
5  int main(int argc, char * argv[]) {  
6  
7      const int i {8};  
8      int j {};  
9      // int *pi {&i}; // error  
10     const int *pi {&i};  
11  
12     pi = &j;  
13 }
```

A constant pointer

```
4  int main(int argc, char * argv[]) {  
5  
6      int i {8}, j{};  
7      int * const pi {&i};  
8  
9      i = 6;  
10     cout << *pi << endl;  
11  
12     pi = &j; // error  
13 }
```

A constant pointer to a constant

```
4  int main(int argc, char * argv[]) {  
5  
6      const int i {8};  
7      const int * const pi {&i};  
8  }
```

Outline

- Pointer
- Pointer and Array
- Dynamic Memory Allocation
- Smart Pointer
- Reference
- The applications of Pointer and Reference in Functions

Pointer and Array

```
#include <iostream>
using namespace std;

int main(int argc, const char * argv[]) {

    double da[] = {1.0, 2.0, 3.0};
    double *p1 = da;
    double *p2 = p1;

    cout << *(da+1) << '\n';
    cout << p1[2] << '\n';
    cout << *(p1+1) << '\n';

    *(p2+1) = 5.0;
    cout << p1[1] << '\n';
}
```

2
3
2
5

Program ended with exit code: 0

iterator

```
5  int main(int argc, char * argv[]) {  
6  
7      vector<int> v;  
8      vector<int>::iterator it;  
9  
10     for (int i = 0; i<10 ; i++) {  
11         v.push_back(i);  
12     }  
13  
14     for(it = v.begin(); it !=v.end(); it++) {  
15         cout << *it;  
16     }  
17     cout << endl << *(it-1) << endl;;  
18  
19 }
```

Practice

- (Practice 1) 取得指標變數與陣列每個元素的位址，並將記憶體畫出來
- (Practice 2) 思考一下字元陣列的複製

Outline

- Pointer
- Pointer and Array
- **Dynamic Memory Allocation**
- Smart Pointer
- Reference
- The applications of Pointer and Reference in Functions

Dynamic Memory Allocation

```
#include <iostream>
using namespace std;

int main(int argc, const char * argv[]) {

    int *pi = new int(5);
    cout << *pi << '\n';
    delete pi;

    int *pia = new int[5];
    *pia = 3;
    *(pia+1) = 7;

    for (int i = 0; i < 5; i++) {
        cout << *(pia+i) << '\n';
    }
    delete [] pia;
}
```

Dynamic Memory Allocation

- <https://shengyu7697.github.io/cpp-nullptr/>

```
1  #include<iostream>
2  using namespace std;
3
4  int main(int argc, const char * argv[]) {
5
6      int *pi = new int {5};
7      cout << *pi << '\n';
8      delete pi;
9      // pi = nullptr;
10
11     if (pi != nullptr)
12         cout << *pi << endl;
13 }
```

Discussion

- malloc / free
- new / delete
- **Memory Leak!**
- std::vector

Memory Leak

```
int *pi = new int(5);  
int *pj = new int(6);  
pi = pj;  
delete pj;
```

Practice

- (Practice 1) Use `std::vector` to implement Linked-List, Queue, and Stack

Outline

- Pointer
- Pointer and Array
- Dynamic Memory Allocation
- Smart Pointer
- Reference
- The applications of Pointer and Reference in Functions

Smart Pointer

- Raw Pointer
 - delete
- **Smart Pointer**

Smart Pointer

- `unique_ptr<T>`
- `shared_ptr<T>`
- `weak_ptr<T>`
- `make_ptr<T>`
- https://kheresy.wordpress.com/2012/03/03/c11_smartpointer_p1/

Smart Pointer

- `unique_ptr<T>`
- `shared_ptr<T>`
- `weak_ptr<T>`
- `make_ptr<T>`
- https://kheresy.wordpress.com/2012/03/03/c11_smartpointer_p1/

Smart Pointer

```
1  #include<iostream>
2  #include<memory>
3  #include<vector>
4  using namespace std;
5
6  int main(int argc, char * argv[]) {
7
8      unique_ptr<int> x( new int {8} );
9      cout << *x << endl;
10     // unique_ptr<int> z = x; // Error
11
12     unique_ptr< vector<int> > y( new vector<int>() );
13     y->push_back(6);
14     cout << y->at(0) << endl;
15 }
```

Arrow operator ->

- <https://www.geeksforgeeks.org/arrow-operator-in-c-c-with-examples/>

Smart Pointer

只能使用 `Shard_ptr`
∴ `unique_ptr` 只能允许
一個指標指向同個
記憶體位置。

```
1  #include<iostream>
2  #include<memory>
3  #include<vector>
4  using namespace std;
5
6  void print(shared_ptr<int> z) {
7      cout << *z << endl;
8  }
9
10 int main(int argc, char * argv[]) {
11
12     shared_ptr<int> x( new int {8} );
13     print(x);
14
15     unique_ptr< vector<int> > y( new vector<int>() );
16     y->push_back(6);
17     cout << y->at(0) << endl;
18 }
```

Smart Pointer (make_shared)

```
1  #include<iostream>
2  #include<memory>
3  #include<vector>
4  using namespace std;
5
6  void print(shared_ptr<int> z) {
7      cout << *z << endl;
8  }
9
10 int main(int argc, char * argv[]) {
11
12     shared_ptr<int> x = nullptr;
13     x = make_shared<int>(8);
14     print(x);
15 }
```

Cyclic Reference

```
1  #include <memory>
2  #include <iostream>
3  class B;
4  class A {
5  public:
6      std::shared_ptr<B> b_ptr;
7      ~A() { std::cout << "A destroyed\n"; }
8  };
9  class B {
10 public:
11     std::shared_ptr<A> a_ptr;
12     ~B() { std::cout << "B destroyed\n"; }
13 };
14 int main() {
15     auto a = std::make_shared<A>();
16     auto b = std::make_shared<B>();
17     a->b_ptr = b;
18     b->a_ptr = a;
19 }
```

weak_ptr

```
1  #include <memory>
2  #include <iostream>
3  class B;
4  class A {
5  public:
6      std::shared_ptr<B> b_ptr;
7      ~A() { std::cout << "A destroyed\n"; }
8  };
9  class B {
10 public:
11     std::weak_ptr<A> a_ptr;
12     ~B() { std::cout << "B destroyed\n"; }
13 };
14 int main() {
15     auto a = std::make_shared<A>();
16     auto b = std::make_shared<B>();
17     a->b_ptr = b;
18     b->a_ptr = a;
19 }
```

weak_ptr

```
1  √ #include <memory>
2    #include <iostream>
3
4  √ int main() {
5      std::shared_ptr<int> shared = std::make_shared<int>(30);
6      std::weak_ptr<int> weak = shared;
7
8  √  if (auto locked = weak.lock()) {
9      // 成功取得 shared_ptr，資源還存在
10     std::cout << *locked << std::endl;
11 }
12
13 }
```


Outline

- Pointer
- Pointer and Array
- Dynamic Memory Allocation
- Smart Pointer
- Reference
- The applications of Pointer and Reference in Functions

Reference

老師提問：
思考和 `int j = i` 的區別

```
4  int main(int argc, const char * argv[]) {  
5  
6      int i = 6;  
7      int &j = i; // j 和 i 是指到同一個記憶體空間, 類似取別名  
8  
9      cout << j << '\n';  
10     j = 5;  
11     cout << i << '\n';  
12 }
```

輸出 5

主要是用在函式的參數傳遞和傳回值上

Outline

- Pointer
- Pointer and Array
- Dynamic Memory Allocation
- Smart Pointer
- Reference
- The applications of Pointer and Reference in Functions

The applications of Pointer and Reference in Functions

- Call by Value
- Call by Address
- Call by Reference

Call by Value

```
#include <iostream>
using namespace std;

void addOne(int x) {
    x++;
}

int main(int argc, const char * argv[]) {

    int x = 5;
    addOne(x);
    cout << x << '\n';
}
```

Call by Address

```
#include <iostream>
using namespace std;

void addOne(int *x) {
    *x = *x + 1;
}

int main(int argc, const char * argv[]) {

    int x = 5;
    addOne(&x);
    cout << x << '\n';
}
```

Call by Address

```
1  #include<iostream>
2  using namespace std;
3
4  void modifyArray(int x[3]) {
5      x[1] = 5;
6  }
7
8  int main(int argc, const char * argv[]) {
9
10     int iarr[3] = {1, 2, 3};
11     modifyArray(iarr);
12     for (int x: iarr) {
13         cout << x << endl;
14     }
15 }
```

Call by Reference

```
#include <iostream>
using namespace std;

void addOne(int &x) {
    x = x + 1;
}

int main(int argc, const char * argv[]) {

    int x = 5;
    addOne(x);
    cout << x << '\n';
}
```


Return Address

```
4  double *avg(int x, int y) {  
5  
6      double result = (x+y)/2.0;  
7      double *avg = new double(result);  
8      return avg;  
9  }  
10  
11  
12  int main() {  
13  
14      double *d = avg(5, 6);  
15      cout << *d << "\n";  
16  }
```

老師提問：
回傳 &result 會怎麼樣？

期中考 會考！

要注意不能傳回局部變數的參考或指標

Return Reference

```
4  double & avg(int x, int y) {
5      double *result = new double((x+y)/2.0);
6      double &ref = *result;
7      cout << "## " << &ref << '\n';
8      return ref;
9  }
10
11 int main(int argc, const char * argv[]) {
12     double & d = avg(5, 6);
13     cout << d << '\n';
14     cout << "@@" << &d << '\n';
15 }
```

老師提問：
如果第 12 行沒有 & 會怎樣？